



Programming in JAVA (CSE2006)

Project Report

Submitted by:

Aklesh Swain

25BAI11184

2. Introduction

Every backend system eventually runs into the same problem: work shows up faster than it can be handled one item at a time, and not all of that work is equally urgent. TaskFlow is my attempt at solving a small, self-contained version of that problem in Java — a simulated job scheduler that takes in a batch of tasks, ranks them by priority, and runs them concurrently across a pool of worker threads.

The goal wasn't just to make something that runs. I wanted to actually use the concurrency tools Java gives you — `ExecutorService`, `PriorityBlockingQueue`, atomic counters — instead of just reading about them. So beyond the scheduling logic itself, TaskFlow also retries failed tasks automatically, logs every state change to a file, and spits out a performance report once a run finishes. The whole thing follows a standard Maven layout under a `com.taskflow` package, and there's a GitHub Actions workflow that compiles the project and runs the test suite on every push, mostly so I'd catch mistakes before they piled up.

3. Problem Statement

Processing tasks strictly one after another, with no sense of priority, wastes time and resources you don't have to waste. Think about a CI/CD pipeline, a background job processor, even something as mundane as a print queue — they all hit the same wall. You need a way to queue work fairly, run as much of it as you can at once without overloading the system, and not fall over completely when one job fails. That's the exact gap TaskFlow tries to fill: a small, single-JVM scheduler that handles priority queueing, concurrent execution, and failure recovery, without needing a distributed system or a database to do it.

4. Functional Requirements

4.1 Task Submission & Priority Queueing (`com.taskflow.queue`, `com.taskflow.model`)

- Tasks are validated on submission (non-blank name, priority ≥ 1 , positive duration).
- Valid tasks are placed into a thread-safe `PriorityBlockingQueue`.
- Tasks are ordered by priority (1 = highest); equal-priority tasks are FIFO-ordered by submission time.
- Invalid submissions raise a custom `InvalidTaskException` rather than corrupting queue state.

4.2 Multi-threaded Task Executor (`com.taskflow.executor`)

- A configurable fixed-size thread pool (`java.util.concurrent.ExecutorService`) processes tasks concurrently.
- Each worker thread continuously pulls the highest-priority available task and executes it.
- A task that fails is automatically retried once before being marked permanently failed.
- Thread-safe counters (`AtomicInteger`) track completed, failed, and retried tasks across all worker threads.

- Graceful shutdown drains in-flight work and releases all threads cleanly.

4.3 Logging & Performance Report Generator (`com.taskflow.logger`, `com.taskflow.report`)

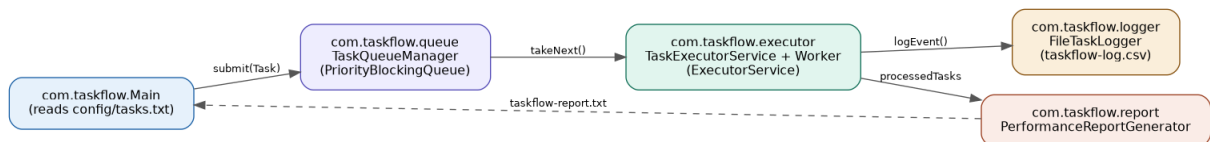
- Every task lifecycle transition (started, completed, failed, retrying) is written to a CSV log file with a timestamp and the worker thread name.
- After a run completes, a performance report is generated: total tasks, completion/failure counts, failure rate, average wait time, average execution time, throughput, and a full per-task breakdown.

5. Non-Functional Requirements

Requirement	How it's addressed
Performance	Thread pool size is configurable to match available CPU cores; queue operations run in $O(\log n)$.
Reliability	Failed tasks get retried once automatically before being marked failed — nothing just silently disappears.
Scalability	Pool size and other tunables live in one place (<code>com.taskflow.util.ConfigLoader</code>) instead of being scattered.
Maintainability	Code is split into focused packages under <code>com.taskflow</code> (model, queue, executor, logger, report, executor).
Error handling	Custom checked exceptions (<code>InvalidTaskException</code> , <code>TaskExecutionException</code>) keep validation failures clear.
Logging / monitoring	Every state transition is logged to a file with timestamp and thread name, so a run can be reconstructed.

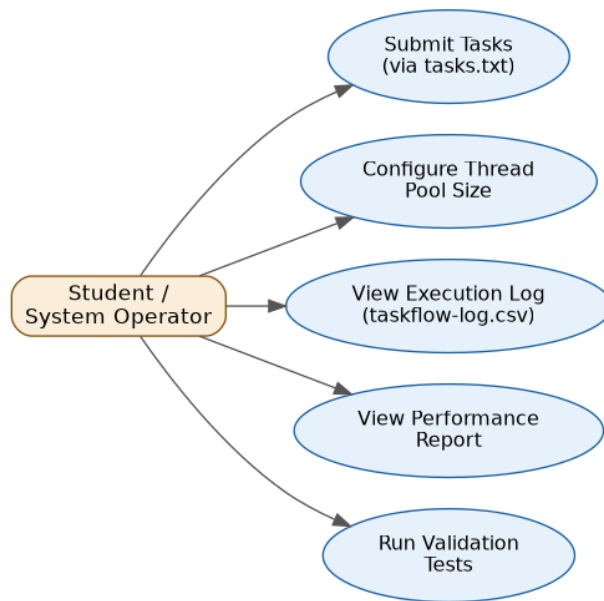
6. System Architecture

The pipeline is deliberately simple, once you see it laid out. `com.taskflow.Main` reads tasks and feeds them into a thread-safe priority queue (`com.taskflow.queue`). A pool of worker threads (`com.taskflow.executor`) pulls from that queue and runs tasks concurrently. Every event along the way gets logged to a file (`com.taskflow.logger`). And once the batch finishes, whatever got recorded is handed off to the report generator (`com.taskflow.report`). Nothing fancy — just five pieces, each doing exactly one thing.

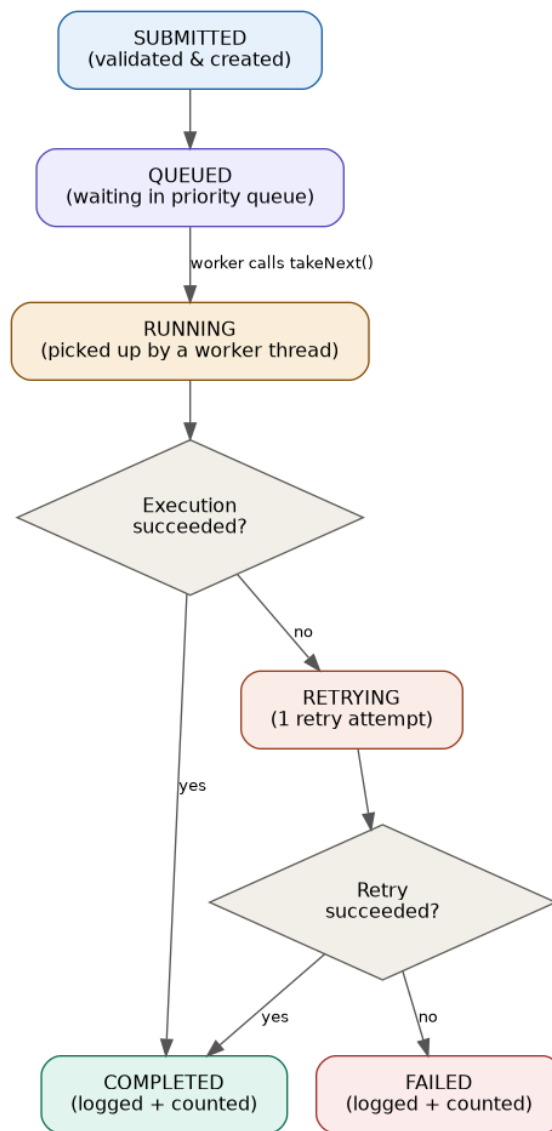


7. Design Diagrams

7.1 Use Case Diagram

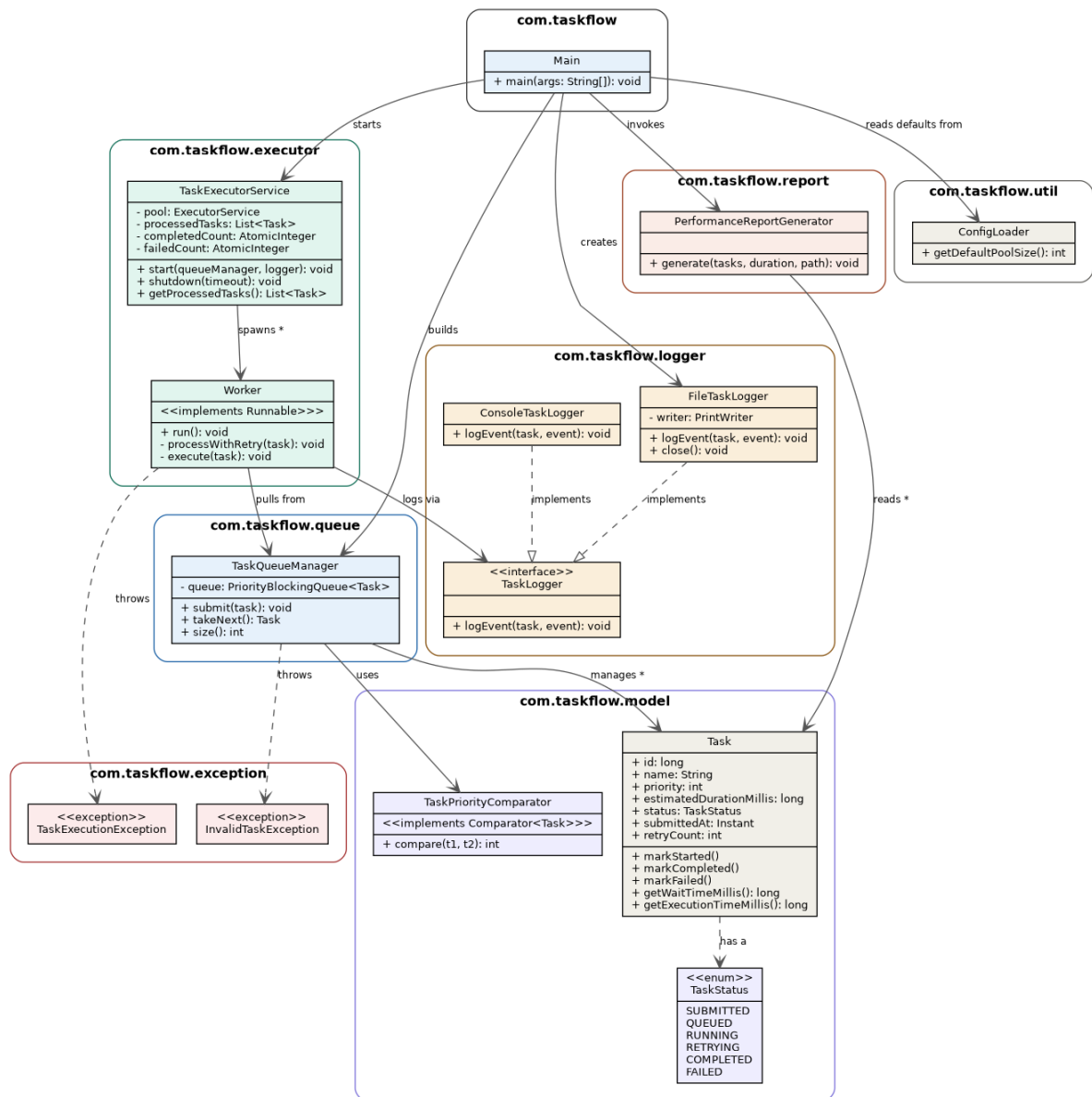


7.2 Workflow / Task Lifecycle Diagram

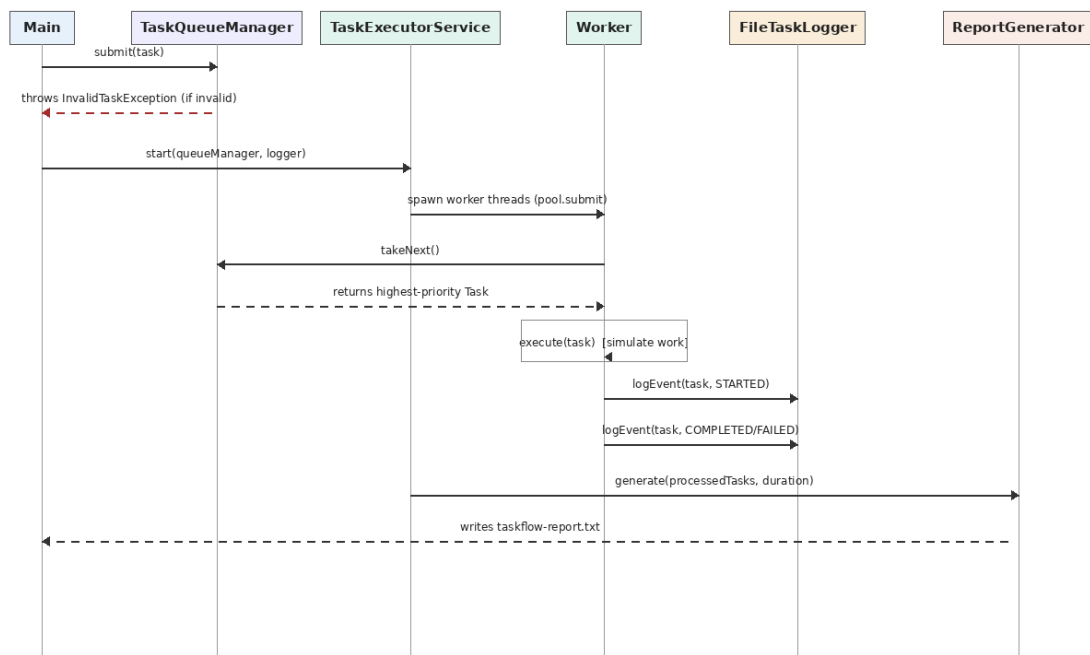


7.3 Class Diagram

Classes here are grouped by their actual package — `com.taskflow.model`, `.queue`, `.executor`, `.logger`, `.report`, `.exception`, `.util` — so this diagram matches the real folder structure under `src/main/java`, not some idealized version of it.



7.4 Sequence Diagram



A note on storage: there's no database here, just CSV log files. So an ER diagram or schema design doesn't really apply to this project — there's no schema to draw.

8. Design Decisions & Rationale

Why a Maven-standard `src/main/java` + `src/test/java` layout with a `com.taskflow` base package?

Because this is what a real Java project looks like. Production code and test code stay separate, every class gets an unambiguous fully-qualified name, and anyone with an IDE can open this and immediately know where things live — no custom compile instructions required.

Why `PriorityBlockingQueue` instead of rolling my own synchronized structure?

No point reinventing it. It's built-in, it's thread-safe, and it supports blocking `take()` calls out of the box — which means worker threads can just wait for new work instead of spinning in a loop checking if anything showed up.

Why a separate `Comparator` instead of making `Task` implement `Comparable`?

Keeps `Task` focused on being data, not on knowing how it should be ordered. If I ever want a different sorting rule later, I swap the comparator — I don't touch the model class at all.

Why retry exactly once, not some configurable number?

Honestly, for a project this size, one retry is enough to prove the failure-recovery logic actually works without overengineering it. `MAX_RETRIES` is still a named constant, so bumping it up later is a one-line change if it ever needs to be.

Why does `Worker` depend on a `TaskLogger` interface instead of just calling `FileTaskLogger` directly?

Dependency inversion, basically — `Worker` shouldn't care how logging happens, just that it does. This also turned out to be genuinely useful during development: I tested the executor with `ConsoleTaskLogger` long before the file-based logger even existed.

Why `CopyOnWriteArrayList` for tracking processed tasks?

Multiple worker threads are appending to this list at the same time, but it only gets read once, at the very end, by the report generator. That's exactly the access pattern `CopyOnWriteArrayList` is built for, and it saves me from writing manual synchronization for something this simple.

Why two separate exception types instead of one generic one?

`InvalidTaskException` and `TaskExecutionException` represent genuinely different problems — one happens before a task even enters the system, the other happens while it's running. Code that catches them needs to react differently in each case, so collapsing them into one exception would have just meant more if-checks downstream.

9. Implementation Details

The whole thing follows a Maven-standard layout (`src/main/java`, `src/test/java`), split across 7 packages under `com.taskflow` — 14 Java files in total. Here's the breakdown:

Package	Classes	Responsibility
<code>com.taskflow</code>	<code>Main</code>	Application entry point; wires all modules together
<code>com.taskflow.model</code>	<code>Task</code> , <code>TaskStatus</code> , <code>TaskPriorityComparator</code>	Core data representation and ordering logic

com.taskflow.queue	TaskQueueManager	Thread-safe submission and retrieval
com.taskflow.executor	TaskExecutorService, Worker	Thread pool management and task execution with retry
com.taskflow.logger	TaskLogger, ConsoleTaskLogger, FileTaskLogger	Pluggable event logging
com.taskflow.report	PerformanceReportGenerator	Post-run statistics and reporting
com.taskflow.exception	InvalidTaskException, TaskExecutionException	Domain-specific error handling
com.taskflow.util	ConfigLoader	Centralized configuration defaults

All the concurrency runs through `java.util.concurrent` — no manual `wait()/notify()`, no hand-rolled synchronized blocks anywhere. `ExecutorService` owns the thread pool, `PriorityBlockingQueue` handles safe access to pending work, `AtomicInteger` tracks the counts without locking, and `CopyOnWriteArrayList` collects results from however many threads happen to be writing to it. Keeping it to built-in thread-safe types instead of writing custom locking logic made the whole thing a lot easier to reason about — and a lot less likely to deadlock.

It builds with Maven (`pom.xml` sets up the compiler, `exec`, and `jar` plugins), but it also compiles fine with plain `javac` against the same `src/main/java` and `src/test/java` trees, for anyone without Maven installed. There's also a GitHub Actions workflow (`.github/workflows/build.yml`) that compiles both source sets and runs the validation suite on every push — mainly so I'd notice immediately if a change broke something, instead of finding out later.

10. Screenshots / Results

10.1 Console Output (com.taskflow.Main)

```
Loaded 10 tasks (0 rejected). Pool size: 4
Processing complete:
Completed: 7
Failed: 3
Retried: 3
Log written to taskflow-log.csv
Report written to taskflow-report.txt
```

10.2 Generated Performance Report

```
=== TaskFlow Performance Report ===
Generated at: 2026-09-13T18:04:24.694325010Z

Total tasks processed : 10
Completed : 7
Failed : 3
Failure rate : 30.00%

Average wait time : 592.40 ms
Average execution time : 976.50 ms
Throughput : 3.55 tasks/sec

Run duration : 2813 ms

=== Per-task Breakdown ===
TaskID,Name,Priority,Status,WaitMs,ExecMs,Retries
7,Refresh cache,1,COMPLETED,46,326,0
2,Send email,1,COMPLETED,42,429,0
4,Cleanup temp files,1,COMPLETED,42,630,0
10,Notify subscribers,1,FAILED,47,1034,1
```

5, Sync inventory, 2, COMPLETED, 472, 901, 0
8, Validate orders, 2, COMPLETED, 672, 1101, 0
6, Compress logs, 3, COMPLETED, 1374, 700, 0
3, Backup database, 2, COMPLETED, 373, 1800, 0
1, Generate report, 3, FAILED, 1082, 1767, 1
9, Archive old records, 3, FAILED, 1774, 1077, 1

10.3 Sample Log Output (taskflow-log.csv, excerpt)

```
timestamp,taskId,taskName,thread,event
2026-09-13T18:04:21.865366997Z,4,Cleanup temp files,pool-1-thread-1,STARTED on pool-1-thread-1
2026-09-13T18:04:21.893346031Z,10,Notify subscribers,pool-1-thread-4,STARTED on
pool-1-thread-4
2026-09-13T18:04:21.893580688Z,2,Send email,pool-1-thread-3,STARTED on pool-1-thread-3
2026-09-13T18:04:21.893694301Z,7,Refresh cache,pool-1-thread-2,STARTED on pool-1-thread-2
2026-09-13T18:04:22.194603440Z,7,Refresh cache,pool-1-thread-2,COMPLETED
2026-09-13T18:04:22.195091363Z,3,Backup database,pool-1-thread-2,STARTED on pool-1-thread-2
2026-09-13T18:04:22.293875267Z,2,Send email,pool-1-thread-3,COMPLETED
2026-09-13T18:04:22.294638558Z,5,Sync inventory,pool-1-thread-3,STARTED on pool-1-thread-3
2026-09-13T18:04:22.402740128Z,10,Notify subscribers,pool-1-thread-4,FAILED (attempt 1):
Simulated random failure for task 10
2026-09-13T18:04:22.403155383Z,10,Notify subscribers,pool-1-thread-4,RETRYING
2026-09-13T18:04:22.494190822Z,4,Cleanup temp files,pool-1-thread-1,COMPLETED
```

11. Testing Approach

I tested this in two rounds. While I was building each module, I wrote a small standalone class just to poke at it in isolation — those live under `com.taskflow.manual` now (`Module1Test`, `Module2Test`, `Module3Test`), and they were genuinely how I caught early bugs before wiring everything together. Once the whole system was in place, I wrote a proper validation suite — `com.taskflow.TaskFlowValidationTest` — with 20 explicit checks covering:

- Priority ordering and FIFO tie-breaking in the queue
- Rejection of invalid task submissions (blank name, non-positive priority/duration)
- Queue size and empty-state reporting as tasks are added and removed
- Task timing calculations (wait time, execution time) at each lifecycle stage
- A full concurrent run through the real thread pool, verifying every task reaches a final state and `completed+failed` counts reconcile with the total submitted

One honest caveat: this isn't a JUnit suite. It's a plain assertion runner, because the machine I built this on had no access to Maven Central to actually pull down a test framework. The `pom.xml` has a comment explaining exactly how to swap it over to JUnit 5 wherever internet access isn't an issue. All 20 checks pass every time I run it, and the same run happens automatically in CI on every push:

Running TaskFlow validation tests...

```
[PASS] Priority ordering: highest priority (1) comes out first
[PASS] Priority ordering: FIFO tiebreak between equal priorities
[PASS] Priority ordering: mid priority (2) comes third
[PASS] Priority ordering: low priority (3) comes last
[PASS] Validation: blank task name is rejected
[PASS] Validation: priority < 1 is rejected
[PASS] Validation: non-positive duration is rejected
[PASS] Queue: reports empty before any submission
[PASS] Queue: size reflects number of submitted tasks
[PASS] Queue: size decreases after takeNext()
[PASS] Queue: reports empty after draining all tasks
[PASS] Timing: wait time is -1 before task starts
[PASS] Timing: execution time is -1 before task completes
[PASS] Timing: status becomes RUNNING after markStarted()
[PASS] Timing: wait time is non-negative once started
[PASS] Timing: status becomes COMPLETED after markCompleted()
[PASS] Timing: execution time is positive after completion
[2026-09-13T18:04:26.105047455Z] Task#13 (Integration job 3) -> STARTED on pool-1-thread-2
[2026-09-13T18:04:26.105661865Z] Task#16 (Integration job 6) -> STARTED on pool-1-thread-1
[2026-09-13T18:04:26.105789905Z] Task#19 (Integration job 9) -> STARTED on pool-1-thread-3
```

```

[2026-09-13T18:04:26.196954165Z] Task#13 (Integration job 3) -> COMPLETED
[2026-09-13T18:04:26.197170205Z] Task#19 (Integration job 9) -> COMPLETED
[2026-09-13T18:04:26.197208297Z] Task#16 (Integration job 6) -> COMPLETED
[2026-09-13T18:04:26.198782537Z] Task#11 (Integration job 1) -> STARTED on pool-1-thread-1
[2026-09-13T18:04:26.199363100Z] Task#14 (Integration job 4) -> STARTED on pool-1-thread-2
[2026-09-13T18:04:26.199572232Z] Task#17 (Integration job 7) -> STARTED on pool-1-thread-3
[2026-09-13T18:04:26.249148274Z] Task#11 (Integration job 1) -> COMPLETED
[2026-09-13T18:04:26.249924537Z] Task#20 (Integration job 10) -> STARTED on pool-1-thread-1
[2026-09-13T18:04:26.250017464Z] Task#14 (Integration job 4) -> COMPLETED
[2026-09-13T18:04:26.250468794Z] Task#12 (Integration job 2) -> STARTED on pool-1-thread-2
[2026-09-13T18:04:26.250044806Z] Task#17 (Integration job 7) -> COMPLETED
[2026-09-13T18:04:26.252736916Z] Task#15 (Integration job 5) -> STARTED on pool-1-thread-3
[2026-09-13T18:04:26.300961694Z] Task#12 (Integration job 2) -> COMPLETED
[2026-09-13T18:04:26.301733678Z] Task#18 (Integration job 8) -> STARTED on pool-1-thread-2
[2026-09-13T18:04:26.302887646Z] Task#20 (Integration job 10) -> FAILED (attempt 1): Simulated
random failure for task 20
[2026-09-13T18:04:26.303157212Z] Task#15 (Integration job 5) -> FAILED (attempt 1): Simulated
random failure for task 15
[2026-09-13T18:04:26.304550160Z] Task#15 (Integration job 5) -> RETRYING
[2026-09-13T18:04:26.304093547Z] Task#20 (Integration job 10) -> RETRYING
[2026-09-13T18:04:26.352098455Z] Task#18 (Integration job 8) -> COMPLETED
[2026-09-13T18:04:26.355994617Z] Task#15 (Integration job 5) -> FAILED permanently: Simulated
random failure for task 15
[2026-09-13T18:04:26.356334388Z] Task#20 (Integration job 10) -> COMPLETED on retry
[PASS] Concurrency: every submitted task reaches a final state
[PASS] Concurrency: completed + failed counts add up to total tasks
[PASS] Concurrency: no task is left in a non-final status

--- Summary ---
Passed: 20
Failed: 0

```

12. Challenges Faced

- **Shutting down cleanly was trickier than expected.** Worker threads block on `queue.take()`, so just calling `shutdown()` didn't actually stop anything — the threads were still sitting there waiting. Had to switch to `shutdownNow()`, which interrupts that blocking call and actually lets each worker's loop exit.
- **Shared counters needed real thread-safety, not just "probably fine."** Multiple workers update `completed/failed/retried` at the same time, and plain `int` fields would've silently dropped updates under contention. `AtomicInteger` fixed that without needing explicit locks.
- **Figuring out where retry logic should actually live.** It needed to sit close enough to the execution call to tell a first failure apart from a second one, but without duplicating the logging and counting code twice. That's what shaped `Worker.processWithRetry()` the way it is.
- **Testing concurrent code deterministically is basically impossible, so I stopped trying to.** Thread scheduling and the simulated random failures mean exact outcomes vary run to run. Instead, the integration test checks invariants that have to hold no matter what order things happen in — `completed + failed` always equals `total submitted`, for instance.
- **Restructuring into Maven layout after the fact was more tedious than hard.** Moving every class into `com.taskflow.*` meant updating package declarations and imports everywhere, consistently. I recompiled and reran the full validation suite after every stage of the move, just to be sure nothing quietly broke along the way.

13. Learnings & Key Takeaways

- Got real hands-on time with `java.util.concurrent` beyond the basic `Thread/Runnable` stuff — using `ExecutorService`, `PriorityBlockingQueue`, `AtomicInteger`, and `CopyOnWriteArrayList` together in one actual system, not just in isolated exercises.
- Thread-safety isn't a checkbox you tick once — it's a property of each specific piece of shared state and how it gets touched. Every shared field here (the counters, the task list, the queue itself) needed its own deliberate thread-safe choice.
- Interfaces like `TaskLogger` make incremental development so much less painful. Being able to swap in a simple console logger meant I could test the executor before the file-based logger even existed yet.
- Once concurrency's involved, you have to test invariants, not exact output — timing and thread ordering just aren't guaranteed to be the same twice.
- A standard Maven layout, even without touching Maven's dependency resolution, makes a project instantly recognizable to anyone who's worked with Java before. It also means growing this into a real dependency-managed project later is a small step, not a rewrite.

14. Future Enhancements

- Persist tasks and results to an actual database (SQLite or MySQL via JDBC) instead of flat files, so runs could be queried and compared historically.
- Make the retry count and backoff delay configurable per task instead of a single fixed constant.
- A lightweight JavaFX or web dashboard to watch queue depth and throughput live, instead of only reading the numbers after the fact.
- Task dependencies — task B waits for task A to finish — using a DAG-based scheduler instead of a flat priority queue.
- Swap the simulated random failure for real task logic (actual file processing, an HTTP call, whatever) so the workload isn't just a stand-in anymore.
- Convert `TaskFlowValidationTest` into a real JUnit 5 suite once Maven Central access isn't a constraint, and wire it back into `pom.xml`'s Surefire config.

15. References

- Oracle Java Documentation — `java.util.concurrent` package: <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/concurrent/package-summary.html>
- Oracle Java Documentation — `ExecutorService` interface
- Oracle Java Documentation — `PriorityBlockingQueue` class
- Apache Maven — Introduction to the Standard Directory Layout
- VITyarthi "Build Your Own Project" — General Project Instructions & Submission Guidelines